# ATCoPE: Any-Time Collaborative Programming Environment for Seamless Integration of Real-Time and Non-Real-Time Teamwork in Software Development

Hongfei Fan Chengzheng Sun
School of Computer Engineering
Nanyang Technological University, Singapore
FANH0003@e.ntu.edu.sg CZSun@ntu.edu.sg

Haifeng Shen
School of Computer Science, Engineering & Mathematics
Flinders University, Australia
haifeng.shen@flinders.edu.au

## ABSTRACT
Real-time collaborative programming and non-real-time collaborative programming are two classes of methods and techniques for supporting programmers to jointly conduct complex programming work in software development. They are complementary to each other, and both are useful and effective under different programming circumstances. However, most existing programming tools and environments have been designed for supporting only one of them, and little has been done to provide integrated support for both. In this paper, we contribute a novel *Any-Time Collaborative Programming Environment* (ATCoPE) to seamlessly integrate conventional non-real-time collaborative programming tools and environments with emerging real-time collaborative programming techniques and support collaborating programmers to work in and flexibly switch among different collaboration modes according to their needs. We present the general design objectives for ATCoPE, the system architecture, functional design and specifications, rationales beyond design decisions, and major technical issues and solutions in detail, as well as a proof-of-concept implementation of the *ATCoEclipse* prototype system.

## Categories and Subject Descriptors
D.2.2 [**Software Engineering**]: Design Tools and Techniques – *computer-aided software engineering (CASE)*; D.2.6 [**Software Engineering**]: Programming Environments – *interactive environments*; H.5.3 [**Information Interfaces and Presentation**]: Group and Organization Interfaces – *computer-supported cooperative work, collaborative computing, synchronous interaction*.

## General Terms
Design, Human Factors.

## Keywords
Any-time, non-real-time, real-time, collaborative programming, seamless integration, compatibility, transparency.


*GROUP'12*, October 27–31, 2012, Sanibel Island, Florida, USA.


## 1. INTRODUCTION
Software development is a complex process which requires sophisticated coordination and collaboration among software engineers with diverse skills, knowledge and expertise [5]. Programming is one important and critical phase during the software development process, in which multiple programmers work collaboratively to transform software requirements into software solutions. The effectiveness of collaboration in programming work is critical to the productivity of programmers and the quality of software products [3][4][5]. In terms of the interactions among collaborating programmers during the programming work, there are two general classes of collaborative programming: *non-real-time collaborative programming* and *real-time collaborative programming*, which are elaborated below.

### 1.1 Non-Real-Time Collaborative Programming
Non-real-time collaborative programming supports multiple programmers to access shared programming artifacts (e.g., source code directories and files), complete individual programming tasks (e.g., editing source code directories and files) independently, and merge their changes on the shared programming artifacts at pre-scheduled stages manually. Non-real-time collaborative programming has been widely applied in modern software industry, and commonly supported by software configuration management (SCM) systems. An SCM system is essentially a version control system for managing any collection of directories and files collaboratively edited by distributed users. Sophisticated version control systems include *Subversion*[1] (SVN) [7], *Concurrent Versions System*[2] (CVS) [2], *IBM Rational ClearCase*[3] [1] and *Git*[4] [11], which commonly support the following working process based on a *copy-modify-merge* model [14][20]:

1) The programmer *checks out* a source code tree of the project which is related to the programming tasks assigned to this individual programmer. A copy of the source code tree is then downloaded from the shared version control repository to the programmer's local workspace.
2) The programmer *modifies* the source code copy in the local workspace to complete his/her programming tasks.

[1] http://subversion.apache.org
[2] http://www.nongnu.org/cvs
[3] http://www.ibm.com/software/awdtools/clearcase
[4] http://git-scm.com

3) The programmer may *merge* the current source code copy in the local workspace with the latest copy in the version control repository in two ways: (1) by issuing an *update* version control operation, other programmers' changes available at the repository are downloaded to the local workspace and merged with the local source code copy; (2) by issuing a *commit* version control operation, the latest version of the local source code copy (as modified by the local programmer) is uploaded to the version control repository for updating the source code copy in the repository, making this local programmer's changes visible and accessible to others. During the *merge* process, the version control system also detects and reports *conflicts* among *concurrent* changes by multiple programmers (e.g., concurrent changes on the same source code file by two collaborating programmers), and leaves the *conflict resolution* to the programmers if the system is unable to reconcile the conflicting changes automatically [13].

With the support of an SCM system, each programmer can complete the programming work independently without being interrupted by others. Such kind of collaborative programming is regarded as *non-real-time* collaboration because neither local changes performed by an individual programmer are immediately propagated and merged with others' copies, nor changes made by others are immediately propagated and merged with the local copy. The local copy is kept *private* until this programmer manually *commits* the local copy into the *public* repository, and in addition, other programmers also have to explicitly perform *update* operations to incorporate the latest committed changes in the repository into their local copies.

## 1.2 Real-Time Collaborative Programming

In contrast, real-time collaborative programming supports a group of programmers to work on shared programming artifacts concurrently in a closely-coupled fashion, in which collaborating programmers' changes on the shared source code copy are instantly propagated and merged [12][17]. In addition, such real-time propagation and merging are achieved automatically without requiring programmers to manually issue version control operations (e.g., *update*, *commit*) as they do in non-real-time collaborative programming. Multiple programmers are allowed to access and edit the same source code directory, and even the content of the same source code file at the same time, as follows:

1) One programmer's editing operations performed on source code directories (i.e., adding/deleting/renaming a directory or file) are immediately propagated and executed at all remote sites, as if the programmer is performing the same operations at all collaborating sites. In other words, changes on source code directories performed by any individual programmer are immediately and automatically made visible to all other collaborating programmers.

2) Multiple programmers are also allowed to work jointly on the same source code file at the same time, and their editing operations performed on the content of the shared file are instantly propagated to others for real-time notification and merging. In other words, operations of each individual programmer on the shared file are automatically performed on all remote copies of the same file, as if collaborating programmers are sitting together and jointly editing the same source code file.

Past studies have found that real-time collaborative programming is capable of accelerating the progress of problem-solving, creating better design and shorter code length, making programmers enjoy the work more, and eventually increasing the productivity of programmers and improving the quality of software products [6][12][17][21][22]. Due to these benefits of the emerging method and technique, numerous real-time collaborative programming tools and environments have been built in recent years, including research prototypes such as *RECIPE* [17], *Collabode* [10] and *Saros*[5] [15], as well as commercial products such as *SubEthaEdit*[6], *beWeeVee*[7] and *VS Anywhere*[8].

## 1.3 Seamless Integration of Real-Time and Non-Real-Time Collaborative Programming

Non-real-time collaborative programming is generally suitable for loosely-coupled, long-duration and pre-scheduled collaboration, which may involve a large number of programmers working on large-size programming modules. In practice, large-scale software projects are commonly decomposed and structured by following well-established design principles (e.g., high modularity, low coupling among modules, separation of concerns), so that each individual programmer is able to work independently on programming modules that are relatively isolated from other programming modules within a large project.

In contrast, real-time collaborative programming is more suitable for a small team of programmers to work on shared and interdependent programming tasks in a closely-coupled fashion. Such kind of work is often unstructured because low-level design, coding and testing activities are mixed within the programming process. It can be started spontaneously and conducted in an ad hoc fashion, and has relatively short durations in a large project.

The main difference between real-time and non-real-time collaborative programming lies on the interactions among programmers. Real-time collaboration enables frequent propagation, notification and merging, while non-real-time collaboration enables infrequent propagation and merging [16]. They are complementary to each other, and both of them are useful and effective under different programming circumstances. However, most existing programming tools and environments have been separately designed for supporting only one of them, with incompatible working processes, user interfaces and working semantics. It is desirable and beneficial to support both of them in one environment, in which programmers can work in any collaboration mode and flexibly switch among different modes according to their needs. In this paper, we propose a novel *Any-Time Collaborative Programming Environment* (ATCoPE) to seamlessly integrate both real-time and non-real-time collaborative programming techniques.

The rest of the paper is organized as follows. In Section 2, general design objectives for ATCoPE are presented as the guidance for system design, functional specifications, and technical implementation. The system architecture and functional design for ATCoPE are presented in Section 3, and major technical issues and solutions of ATCoPE are discussed in Section 4. The implementation of the *ATCoEclipse* prototype system is presented in Section 5, followed by conclusions and future work in Section 6.

[5] http://www.saros-project.org

[6] http://www.codingmonkeys.de/subethaedit

[7] http://www.beweevee.com

[8] http://www.vsanywhere.com

## 2. GENERAL DESIGN OBJECTIVES

In this section, four design objectives for ATCoPE are presented as the general guidance for technical design and implementation.

***Design Objective 1: Compatibility and transparency in supporting non-real-time collaborative programming***

ATCoPE is compatible with existing non-real-time collaborative programming tools and environments in terms of user interfaces, functionalities and features, working processes, and working semantics. It achieves such compatibility by transparently incorporating existing single-user programming environments and non-real-time collaboration supporting tools, without any change to the source code of existing systems.

For end-users (programmers) of ATCoPE, compatibility means that they can continue to use familiar single-user programming environments (e.g., *Microsoft Visual Studio*[9], *Eclipse*[10]) and non-real-time collaboration supporting tools (e.g., CVS, SVN) with the same skills, knowledge and experience as before, while enjoying emerging real-time collaboration capabilities. For researchers and system builders of ATCoPE, transparency means that they can achieve conventional single-user programming functionalities and non-real-time collaboration supporting capabilities by reusing and incorporating existing tools and environments, without reinventing them from scratch.

***Design Objective 2: Capability of supporting advanced real-time collaborative programming***

ATCoPE supports multiple programmers to freely and concurrently work in a shared collection of source code directories and files for the same project at the same time. Under the real-time collaboration mode, multiple programmers can create, delete and update any directory and file for the shared project; they can instantly see others' updates to the shared source code directories and files in real-time. ATCoPE resolves operation conflicts and ensures consistency of shared data in the face of concurrent manipulation, and provides advanced workspace awareness support.

***Design Objective 3: Capability of supporting any-time collaborative programming***

ATCoPE supports not only simultaneous real-time and non-real-time collaboration sessions for different projects, but also any-time collaboration sessions for the same project, which may consist of real-time and non-real-time collaboration sessions at the same time. An any-time collaboration session comes into existence as long as there are multiple programmers working in both real-time and non-real-time collaboration sessions for the same project. In an any-time collaboration session, programmers may work individually, collaborate with others in a conventional non-real-time fashion, and work in a closely-coupled real-time fashion at the same time. ATCoPE also supports real-time collaborating programmers to perform conventional non-real-time collaboration functions (e.g., *check-out*, *update* and *commit*) for the shared project and collectively resolve non-real-time collaboration conflicts in a real-time collaboration fashion. Moreover, ATCoPE allows programmers to switch among different collaboration modes and sessions flexibly according to their collaboration needs.

[9] http://www.microsoft.com/visualstudio

[10] http://www.eclipse.org

***Design Objective 4: High performance and scalability***

ATCoPE provides end-users with high local responsiveness (as responsive as single-user programming tools and environments), fast remote notification and merging for real-time collaborating programmers, and efficient bandwidth usage for communications among any-time collaborating programmers. ATCoPE also maintains good system performance as the increase of the number of collaborating programmers and the number of simultaneous active collaboration sessions for large-scale software projects.

## 3. SYSTEM ARCHITECTURE AND FUNCTIONAL DESIGN

### 3.1 ATCoPE System Architecture

To achieve the design objectives, the ATCoPE system architecture is proposed in Figure 1, and the functionalities of its key components are also specified. The ATCoPE system consists of a server and multiple clients connected by communication networks.

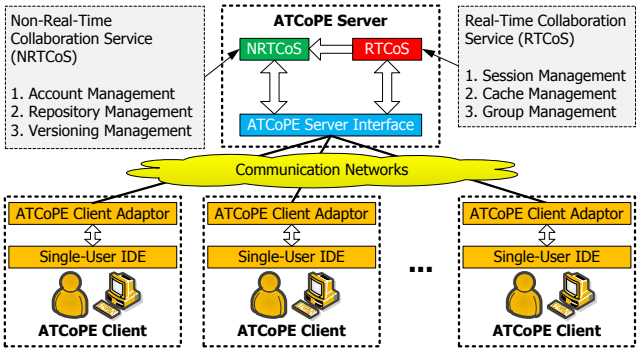


**Figure 1. The ATCoPE system architecture.**

The ATCoPE Server contains a *Real-Time Collaboration Service* (RTCoS) component and a *Non-Real-Time Collaboration Service* (NRTCoS) component, as well as a uniform *ATCoPE Server Interface* for any-time collaboration service (Design Objective 3). The NRTCoS component is responsible for account management, source code repository management, and versioning management, which are commonly supported by version control systems. The NRTCoS component incorporates these functionalities from existing systems (Design Objective 1). The RTCoS component is responsible for providing advanced real-time collaboration services (Design Objective 2), including real-time collaborative programming session management, project cache management, and group membership management. The RTCoS makes use of the NRTCoS for conventional account and repository management, as well as other non-real-time collaboration services (Design Objective 3).

An ATCoPE Client provides an integrated development environment (IDE) with comprehensive facilities for coding, compilation and debugging, as well as various tools (e.g., class browser). The IDE is also integrated with a version control client for supporting non-real-time collaboration. Such features are commonly supported by conventional IDE products (e.g., Eclipse), so the ATCoPE Client can incorporate existing tools and systems without reinvention (Design Objective 1). While preserving all conventional IDE functions and interface features, the ATCoPE Client also provides additional features for real-time collaborative programming. The *ATCoPE Client Adaptor* is designed to transparently convert a conventional IDE into an ATCoPE Client, without any change to the source code of the IDE, which can be achieved by following

the *Transparent Adaptation* (TA) approach proposed in prior work [19]. Among various tools and facilities encapsulated in an IDE, the current design of the ATCoPE Client focuses on extending source code editing tools for supporting any-time (both real-time and non-real-time) collaboration (Design Objective 3).

## 3.2 Functional Design of ATCoPE

### *3.2.1 Logging into the ATCoPE System: Account and Repository Management*

Like working with a conventional version control system such as SVN, a programmer needs a user account to use the repository, version control, and other collaboration services in the ATCoPE system. Within the ATCoPE Server, the NRTCoS component is responsible for incorporating source code repository and version control functions from existing tools, and the ATCoPE Client Adaptor is responsible for keeping the process of using user accounts to log into the ATCoPE system the same as that of using a conventional IDE with an integrated version control client. For end-users, the ATCoPE login process for both real-time and non-real-time collaboration is the same: using the same user account and interface to access the same source code repository managed by the same version control system. After logging into ATCoPE, the programmer can proceed to access and browse the source code trees granted to his/her account in any way.

### *3.2.2 Creating a Session: Start of Collaboration*

After logging into ATCoPE, a user may select any source code file or sub-tree of files, and then issue a *check-out* command to download the selected files to the local workspace, which triggers the creation of an ATCoPE session. The difference between real-time and non-real-time collaboration sessions comes into existence when different collaboration options are chosen at the time of checking out files (see Section 5.2).

If the "*non-real-time*" option is chosen, the selected files are downloaded from the NRTCoS repository to the ATCoPE Client, without any interaction with the RTCoS component. The notion of a non-real-time session is *implicit* in the sense that no explicit session record is created in the ATCoPE Server.

If the "*real-time*" option is chosen, the selection of the files to be checked out, together with the user account authentication data, is transmitted to the RTCoS component to create a real-time session record, which includes the group membership information (initially the first user's account name and authentication data) and a cache of the source code copy being checked out. The source code copy associated to this session is then downloaded and duplicated at the client. The creation of a real-time session is transparent to the NRTCoS: the check-out process in the version control system is always the same, regardless of whether a real-time session is created or not. Each explicit real-time session managed by the RTCoS component corresponds to one implicit non-real-time session in the NRTCoS version control system, and such correspondence facilitates close integration and easy switching among real-time and non-real-time sessions (see Section 4.3).

For end-users, the creation of a real-time or non-real-time session in the APCoPE system is nearly the same, except for ticking a different option at the time of checking out files from the version control repository. What happens at the server-side is invisible to the user, but some distinctive visual clues must be provided at the ATCoPE client interface to differentiate whether the current session is real-time or non-real-time (see Section 5.2).

### *3.2.3 Joining a Session: Dynamic Session Membership Management*

In the ATCoPE system, a user is provided with not only a conventional repository interface for browsing and checking out files to create new sessions, but also a list of existing real-time sessions available for the programmer to join (see Section 5.2). Joining a non-real-time session happens implicitly as long as the programmer checks out files from the version control repository. Differently, to join a real-time session, the user needs to request permission. In the simplistic case, a user may automatically become a new member of a selected existing real-time session if the session is open for anyone who holds a valid ATCoPE account. In addition, more sophisticated group membership management policies and strategies are possible. For example, the creator of a real-time session may give permissions to a specific group of ATCoPE users, and/or require late-comers to make explicit requests and grant permissions on a case-by-case basis. Once the permission is obtained, the user will join the real-time session under the control of a distributed join-protocol for ensuring consistency (to be discussed in Section 4.2). Upon joining, the new session member is recorded in the RTCoS component and the source code copy associated to this session is downloaded and duplicated at the ATCoPE Client of this new member. At the same time, all existing members of the real-time session are notified of the joining of the new member.

### *3.2.4 Working in a Session: Features of Any-Time Collaboration*

In a non-real-time session, a programmer works on source code copies inside the private workspace; changes to those copies are local and invisible to the public until the local programmer manually *commits* the changes back to the version control repository. To incorporate changes made by others, a programmer has to explicitly issue an *update* operation, which merges the latest version of the shared source code copy at the version control repository with his/her local version. The *update*/*commit* process may involve conflict detection and conflict resolution, as conflicts may occur in the face of concurrent changes on shared files.

In a real-time session, a programmer also works on source code copies inside the local workspace, but changes to those copies become visible automatically and instantly to other collaborating programmers without manually executing *update*/*commit* operations. Especially, multiple programmers may jointly edit one file at the same time, thus creating an internal real-time collaborative editing session. To differentiate these real-time sessions, we use the term *real-time project session* to refer to a real-time session created when checking out files from the version control repository, and use the term *real-time file session* to refer to a real-time collaborative editing session created when multiple programmers are editing the same file inside a real-time project session.

During a real-time session, programmers may freely and concurrently perform two types of editing operations: (1) *file-level editing*: to edit the content of any source code file by inserting and deleting texts; and (2) *project-level editing*: to create, delete and rename directories and files for the shared project. The timeliness of propagating changes to other collaborators depends on the nature and need of the collaborative work, as specified below:

1) When multiple programmers are working on the same source code file in a real-time file session, changes made by one programmer are propagated to all members within the same file

session *instantly*, but to other members who are outside the file session but within the same project session *at the time of saving the file*. Conflicts caused by concurrent editing operations on the file content are automatically resolved by underlying consistency maintenance techniques such as the *Operational Transformation* (OT) [18] and *Dependency-based Automatic Locking* (DAL) [8]. The delayed propagation to those members outside the file session is reasonable as they are not working on or interested in the latest content of the file at the moment, and this helps achieve good performance and efficient use of communication bandwidth (Design Objective 4).

2) When multiple programmers are concurrently changing the shared source code copy by creating, deleting and/or renaming directories and/or files, their changes are *instantly* propagated to all members within the same project session. If concurrent editing operations target the same directory/file, operation conflicts may occur and will be automatically resolved by certain conflict resolution measures provided in the underlying system. The conflict resolution mechanisms and policies have been designed for ensuring consistency of the shared source code copy and preserving conflicting operations' effects as much as possible. For example, if two concurrent *Create* operations use the same name for the directory/file to be created, a reasonable conflict resolution result is to keep both but rename them properly, in order to preserve both *Create* operations' effects while ensuring consistency.

In addition to project-level and file-level editing operations with effects in the scope of the same real-time session, a programmer may also issue version control operations (e.g., *update*, *commit*) to merge the source code copy of the real-time session with the latest committed versions from other non-real-time collaborators in the version control repository. All interactions between a real-time session and the version control system are performed in the name of one user who created the real-time session. From the version control system's perspective, a real-time session is merely a single user, regardless of how many members are actually involved in the session. If conflicts with other non-real-time collaborators are detected and reported in processing *update* or *commit* operation, real-time collaborators may jointly resolve them: they may separately resolve conflicts in different files, or jointly resolve conflicts in the same file in a real-time file session. Such kind of close interaction among real-time and non-real-time collaborators is a unique any-time collaboration feature of the ATCoPE system.

### *3.2.5 Leaving and Terminating a Session: Completion of Collaborative Work*

Upon completion of collaborative work, a programmer may leave a session at any time. Leaving a non-real-time session is implicit by simply issuing a *commit* operation to merge the local source code copy with the latest copy in the version control repository. Leaving a real-time session can be initiated by a session member under the control of a distributed leave-protocol that flushes all local changes at the leaving site to other existing sites and notifies them of the leaving event, and the corresponding session membership record will be updated in the RTCoS component. After the last session member has left a real-time session, this session will be terminated, which results in the removal of the whole session record from the RTCoS component. Before the termination of a real-time session, the source code copy in the RTCoS cache will be committed back to the version control repository, which signifies leaving the corresponding non-real-time session.

### *3.2.6 Switching among Collaboration Modes and Sessions: Flexible Any-Time Collaboration*

With ATCoPE, a programmer may switch among real-time and non-real-time sessions freely. For example, a programmer may check out a collection of source code files from the version control repository and work individually in a non-real-time collaboration fashion (by using *update* and *commit* operations) for a while. At a later moment, this programmer may wish to have more closely-coupled interaction with some collaborators for solving certain problems, and s/he can simply click a button at the client interface (see Section 5.2) to create a real-time session based on the local source code copy. Internally, this will trigger the uploading of the local source code copy to the RTCoS cache and the creation of a new real-time session, which will become visible to other programmers who have currently (or later) logged into the system. Conversely, a real-time session member may quit the current real-time session and continue to work on the same project (based on his/her local source code copy) as a non-real-time collaborator.

# 4. TECHNICAL ISSUES AND SOLUTIONS

In this section, several major technical issues and solutions are presented for supporting the ATCoPE system as designed in previous sections.

## 4.1 Consistency Maintenance

To achieve responsive and unconstrained real-time collaboration in high latency communication networks like the Internet, the real-time collaborative source code editor in the ATCoPE Client has been designed with a *replicated architecture* where the shared source code copy is replicated at each collaborating site to allow local operations to be responded and executed quickly (Design Objective 4). When a programmer is editing any shared directory or file in a real-time session, editing operations issued are immediately performed on the local replica (thus achieving responsiveness), and then captured by the ATCoPE Client Adaptor and instantly propagated to remote sites for execution (thus achieving real-time notification and merging). Such kind of collaborative editing is unconstrained in the sense that programmers can freely and concurrently access and update any part of the shared programming artifacts at any time.

Under the replicated architecture, consistency maintenance becomes an essential requirement and technical challenge: after all editing operations are propagated and executed at all collaborating sites within a real-time session, the distributed replicas of the source code directories and files should be identical across all collaborating sites. As there are two types of real-time sessions (i.e., real-time project session with project-level editing operations and real-time file session with file-level editing operations), suitable consistency maintenance techniques have been devised and applied for each of them respectively, as follows.

### *4.1.1 Consistency in Real-Time Project Sessions*

In a real-time project session, each project-level editing operation is immediately performed at the local replica, automatically captured by the ATCoPE Client Adaptor, and instantly propagated, via the *real-time session manager* inside the ATCoPE Server, to all other collaborating sites for remote execution. As mentioned in Section 3.2.4, concurrent editing operations may cause conflicts. One possible approach to solving this problem is to use a locking-based protocol to serialize concurrent operations. Under this approach, one site must obtain the permission token from the central

coordinator before performing any project-level editing operation, and the token must be returned back to the coordinator after the operation has been done locally and executed at remote sites. This approach achieves consistency by prohibiting concurrent project-level editing operations, thus eliminating the possibility of operation conflicts. However, it is too restrictive for real-time collaborative work (which often requires unconstrained interaction), and may incur high computation and communication overhead because each project-level editing operation needs a costly global synchronization in high-latency communication networks.

To achieve high performance and scalability (Design Objective 4), an optimistic approach to concurrency control has been adopted for project-level editing operations: programmers are allowed to perform project-level editing operations without any delay, and the system automatically detects and resolves operation conflicts at remote sites. To detect and resolve conflicts among project-level editing operations, the following techniques are applied. Firstly, a scheme based on state vectors [18] is devised to time-stamp each project-level editing operation with a state vector, which is used to derive causal relationships (one necessary condition for conflicts) among operations. Secondly, a history buffer is maintained at each site to save all project-level editing operations that have been executed at the site. When a remote operation arrives and becomes causally ready for execution, operations saved in the history buffer are scanned to check whether they are concurrent with the newly arrived operation by comparing their timestamps. If an executed operation is found to be concurrent with the newly arrived one, detailed operation information (such as the operation type and the pathname of the targeted directory/file) is further examined to check whether the concurrent operations are conflicting with each other. In case that a conflict is detected, the corresponding conflict resolution strategy is applied.

### *4.1.2 Consistency in Real-Time File Sessions*

In a real-time file session, consistency maintenance is concerned with the textual content of the shared source code file that is being collaboratively edited. After all file-level editing operations are executed at all sites within a real-time file session, the textual content of the shared source code file in each local replica should be identical. Similar to a project-level editing operation, a file-level editing operation is also immediately applied on the local replica of the shared file, automatically captured by the ATCoPE Client Adaptor, and then instantly propagated, via the ATCoPE Server, to all other collaborating sites within the same real-time file session. Concurrent file-level editing operations may also conflict with each other due to the positional shifting effects of textual editing operations, which is a well-known problem and can be resolved by using the OT technique contributed in prior work [18]. The OT technique has been invented to maintain consistency without restricting user interactions, which is able to achieve two key consistency requirements: (1) *convergence*: all replicas of the shared document must be identical after executing the same group of operations; and (2) *intention preservation*: the effect of an operation in all replicas must be the same as its effect at the local replica. With the support of other distributed computing techniques, a real-time collaborative editing system also ensures *causality preservation* to ensure that editing operations are always executed in their *cause-effect* order at all collaborating sites. Technically, the OT technique transforms parameters of concurrent operations to compensate the positional shifting effects (in the domain of plain text editing), so that their execution in different orders can produce consistent and intended results.

## 4.2 Join-Protocol for Dynamic Membership

ATCoPE supports dynamic membership in real-time sessions where collaborating sites can join and leave a session at any time during collaborative programming. To accommodate late-comers at any time in a reliable way, a distributed join-protocol has been designed, with the following communication messages:

- JOIN: sent from a new client to the session manager to request joining an existing real-time session.
- START: sent from the session manager to all existing clients in the real-time session to inform the start of a join-protocol procedure for accepting a new client.
- READY: sent from an existing client to the session manager to inform its readiness for entering the *quiescence* state.
- FINISH: sent from the session manager to all clients (including the new client) to inform the completion of the procedure.

Major steps of the join-protocol are listed and described below. In addition, an example is presented in Figure 2 to illustrate how the join-protocol works step-by-step in a real-time session with two existing clients for accepting a new client to join.

- When a new client attempts to join a real-time session, it sends a JOIN message to the real-time session manager of the ATCoPE Server. After that, this client is blocked (with no UI operation allowed) until the completion of the procedure.
- Upon receiving a JOIN message from a new client, the session manager broadcasts a START message to all existing clients in the real-time session.
- Upon receiving a START message sent from the session manager, a client completes all ongoing operations and sends back a READY message to the session manager. After this, the client is blocked (with no UI operation allowed), but still allows the execution of remote operations, which is required by the synchronization protocol.
- Upon receiving READY messages from all existing clients, the session manager broadcasts a FINISH message to all (existing and new) clients. At the same time, the session manager registers the new client in the session's active membership list.
- Upon receiving a FINISH message, a client is unblocked for resuming normal editing work. Particularly, the new client will also receive the latest source code copy of the project from the server, together with the FINISH message.


**Figure 2. Join-protocol in real-time sessions.**

Under the assumption that all communications between clients and the server are performed in FIFO channels (e.g., TCP connections), a session will enter the quiescent state when all clients have received the FINISH message. In a quiescent state, there is no message in transition, and all sites must have executed the same collection of operations and hence have consistent copies of source code. At the end of the join-protocol procedure, the new comer will receive the latest source code copy of the project, and all sites will resume work as if a fresh session is started. In case

that multiple clients concurrently issue joining requests for the same session, they are serialized by the session manager to avoid inconsistency and complication. When a joining request arrives at the session manager, if one join-protocol procedure for the same real-time session is in progress, the coming request will be queued until the ongoing procedure is completed. However, concurrent joining for different sessions can be processed in parallel. For a leaving request from an existing client, the session manager simply broadcasts a LEAVE message to other existing clients to inform the event, removes the site from the session's active membership list, and closes the network connection with the client.

## 4.3 Incorporating Non-Real-Time Collaboration Service in ATCoPE

Based on the general design objectives, ATCoPE should be compatible with existing single-user programming environments and non-real-time collaboration supporting tools, and achieve such compatibility by transparently incorporating existing IDEs and version control systems. The transparent integration with existing single-user programming environments and converting them into multi-user real-time collaborative programming environments can be achieved by taking the TA approach proposed in prior work [19]. The transparent integration with non-real-time collaboration service involves multiple components inside the ATCoPE system, which collectively achieve the following objectives: (1) incorporating conventional and existing non-real-time collaboration service supporting tools for non-real-time collaborative programming sessions in ATCoPE; and (2) bridging non-real-time collaboration service and real-time collaborative programming sessions. In the following parts, we present how various ATCoPE components collaborate with each other to achieve these objectives.

As shown in Figure 3, for a non-real-time collaborative programming session, an ATCoPE Client issues version control operations (e.g., *check-out*, *update*, *commit*) in the same way as working in a conventional IDE integrated with a version control client. The ATCoPE Server Interface processes these operations by simply passing them to the NRTCoS component in the ATCoPE Server, which in turn invokes public interfaces of existing version control systems to execute these operations. At the beginning of a non-real-time session, the source code copy is checked out from the NRTCoS repository to the local workspace of the ATCoPE Client. In the face of *update*/*commit* operations, the source code copy is either downloaded from the NRTCoS repository to the ATCoPE Client's local workspace (by *update*), or uploaded from the local workspace to the NRTCoS repository (by *commit*). In summary, the ATCoPE client user issues version control operations by using the same process and user interface, executing operations with the same semantics, and achieving the same results as working with a conventional IDE and version control client.

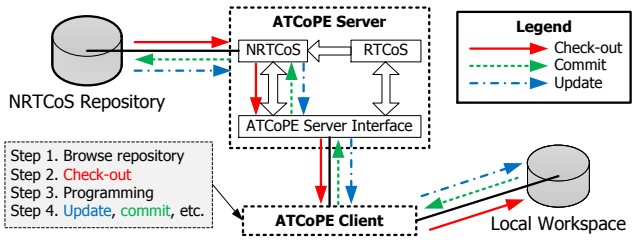


**Figure 3. Incorporating non-real-time collaboration service for non-real-time collaboration sessions in ATCoPE.**

As illustrated in Figure 4, to support the use of the non-real-time collaboration service from real-time collaboration sessions, each real-time session is associated with a *Non-Real-Time Client Proxy* in the RTCoS component, which bridges the non-real-time collaboration service and the real-time collaborating programmers. Inside the ATCoPE Server, each real-time session has a corresponding *session cache* for storing the latest copy of the project's source code as a shared repository for the period of the real-time session. When a real-time session is created by an ATCoPE user, the source code copy is checked out from the NRTCoS repository to the corresponding session cache for the newly created real-time session in the RTCoS. The source code copy is then downloaded from the RTCoS Session Cache to the client's local workspace via the ATCoPE Server Interface. Similarly, when a new site joins the real-time session, the latest copy of the project's source code is transmitted from the RTCoS Session Cache to the local workspace of the new client, upon completion of the join-protocol procedure as presented in Section 4.2. During a real-time session, programmers concurrently perform various editing operations on the shared source code copy, and changes are propagated to other sites instantly and saved to the RTCoS Session Cache as well.

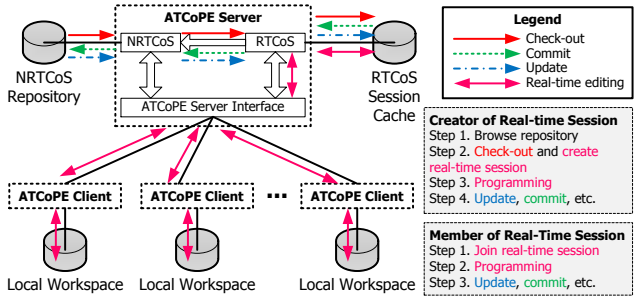


**Figure 4. Bridging non-real-time collaboration service and real-time collaboration sessions in ATCoPE.**

Whenever a member of the real-time session issues an *update* or *commit* operation, this operation is transmitted to the corresponding Non-Real-Time Client Proxy inside the RTCoS component, which relays these version control operations (and associated data) to the NRTCoS version control system. In processing an *update* operation, the latest copy in the NRTCoS repository is downloaded and merged with the current copy stored in the session cache, and the updates are then downloaded to all clients within the real-time session. Conversely, in processing a *commit* operation, the latest source code copy stored in the session cache is committed to the NRTCoS repository. The Non-Real-Time Client Proxy represents the real-time session's creator and acts as a single client of the NRTCoS, so the credentials of the session creator must be saved in the RTCoS session manager at the time of session creation for facilitating authentication when communicating with the NRTCoS component.

In processing non-real-time version control operations within a real-time session, the proxy must also ensure consistency of the shared source code copy among all session members. This has been achieved by using synchronization protocols to force the session to reach a quiescent and consistent state. For supporting *update* and *commit* operations in real-time collaborative programming sessions, synchronization protocols similar to the join-protocol have been devised, which simply replace the JOIN message with an UPDATE or COMMIT message. When the session reaches a quiescence state, the shared source code copy must be consistent across all sites, and this latest copy is committed to the

NRTCoS repository, or merged with the downloaded copy from the NRTCoS repository by the *update* command. In executing either *update* or *commit* operation, conflicts may be detected and reported by the NRTCoS version control system, and the resolution of those conflicts can be carried out in a real-time collaboration fashion (using ATCoPE real-time collaboration service). Moreover, in the face of concurrent version control operations issued by multiple active members in the same real-time session, they are serialized by the session manager.

# 5. PROTOTYPE IMPLEMENTATION AND PRELIMINARY EVALUATION

To validate the feasibility of ATCoPE as well as the functional design and technical solutions, a prototype named *ATCoEclipse* (*A*ny-*T*ime *Co*llaborative Programming with *Eclipse*) has been implemented, which realizes the system architecture, techniques, and solutions derived from the research, and serves as a proof-of-concept for ATCoPE.

To achieve transparency and compatibility with existing single-user programming environments, the ATCoPE Client has been implemented as a plug-in with the popular Eclipse IDE, using the TA approach and the *Generic Collaboration Engine* (GCE) contributed in prior work [19]. To achieve compatibility with existing version control systems for non-real-time collaboration service, the SVN has been adopted for supporting conventional version control functionalities in the ATCoEclipse system.

These design decisions have been made based on the following reasons. Firstly, the Eclipse IDE is an open platform with a core runtime engine and various subsystems as plug-ins, allowing developers to add extensions easily. SVN is also a free and open source system which provides extensibility and reusability. Secondly, the Eclipse IDE contains a rich set of popular plug-ins that provides not only useful features but also programming interfaces for other plug-ins to utilize. Similarly, SVN also provides rich interfaces for supporting version control functions that can be utilized by external applications in communicating with the SVN service (e.g., sending SVN commands). Last but not least, the Eclipse IDE is widely used by a large number of users in software industry and academic communities due to its free, open-source, and extensible properties, and similarly, SVN is also a sophisticated and popular system with a wide range of users. Large popularity of Eclipse and SVN provides great opportunities for usability study and evaluation.

## 5.1 ATCoEclipse System Architecture

As presented in Figure 5, the ATCoEclipse system architecture is an instance of the generic ATCoPE system architecture in Figure 1, with the ATCoPE Client instantiated by the *ATCoEclipse Client*, and the ATCoPE Server instantiated by the *ATCoEclipse Server*. The ATCoEclipse system takes a client-server communication structure where multiple ATCoEclipse Clients are connected to the ATCoEclipse Server via the Internet.

At the client-side, the *ATCoEclipse Client Adaptor* transparently converts the existing single-user Eclipse source code editor into an advanced multi-user real-time collaborative source code editor. Within the ATCoEclipse server-side, the SVN Service is incorporated as the non-real-time collaboration service. The *RTCoServer* (*Real-Time Collaboration Server*) component contains various functional modules for collectively offering real-time collaboration service, and transparently incorporates the SVN Service for supporting non-real-time collaboration functionalities in real-time collaboration sessions. The *ATCoEclipse Server Interface* provides a uniform any-time collaboration interface for processing both real-time and non-real-time collaboration requests and commands from all ATCoEclipse Clients.

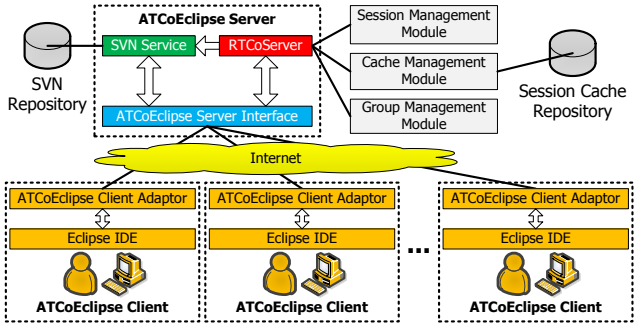


**Figure 5. The ATCoEclipse system architecture.**

## 5.2 User Interface Design of ATCoEclipse

Figure 6 presents the user interface for a programmer to initialize programming work with ATCoEclipse, which is designed according to functional specifications in Section 3.2.1, 3.2.2 and 3.2.3. In the upper-left part of the dialog box, the programmer specifies the SVN repository to access and the user account (with username and password) used in the SVN system, which is similar to the authentication process for using a conventional SVN client. Upon successful authentication (performed by the SVN Service via the ATCoEclipse Server Interface), the source code directories and files located at the specified location are retrieved and displayed as a tree at the ATCoEclipse Client, as shown in the lower-left part of Figure 6.

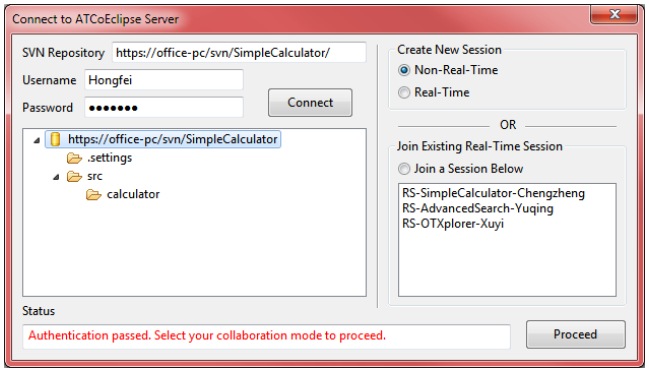


**Figure 6. Initialization of programming work.**

If the programmer wants to create a new collaboration session, s/he can browse this source code tree and specify a location to check out, which acts similarly as a conventional version control client integrated with single-user programming environments. After specifying the source code directories and files to check out, the programmer proceeds to choose the collaboration mode in the right panel. The programmer is provided with two options for the collaboration mode: (1) non-real-time collaboration, which will directly check out the source code copy from the SVN repository to the local workspace; and (2) real-time collaboration, which will trigger the ATCoEclipse Server to create a new session (with this user as the session owner and creator), check out source code copy from the specified location in the SVN repository to the session cache at the ATCoEclipse Server, and then transmit the source code copy to the programmer's local workspace.

Alternatively, the programmer can also join an existing real-time collaborative programming session that is available in the system. In the lower part of the right panel in Figure 6, a list is displayed to show all existing real-time sessions that are available for this programmer to join. In this figure, there are three real-time sessions that are available at the moment: (1) *RS-SimpleCalculator-Chengzheng*, which is a real-time session for the project named *SimpleCalculator* created by the programmer named *Chengzheng*; (2) *RS-AdvancedSearch-Yuqing*, which is a real-time session for a different project named *AdvancedSearch* created by another programmer named *Yuqing*; and (3) *RS-OTXplorer-Xuyi*, which is a real-time session for another project named *OTXplorer* created by a programmer named *Xuyi*. By simply choosing the collaboration mode in the right panel, the corresponding collaboration session is created or joined, leading the user to the programming work.

Figure 7 presents the main user interface of the ATCoEclipse Client when a programmer is conducting programming work in a real-time collaboration session. Similar to the single-user Eclipse IDE, ATCoEclipse's programming interface contains a *package explorer* as shown in the left panel, but additional real-time collaboration features have been embedded: when the programmer issues any operation in the package explorer to create, delete or rename a directory or file, this operation is instantly propagated and replayed at all other active sites within the same real-time session. Similarly, other session members' changes to the source code tree are also performed in this package explorer in real-time.

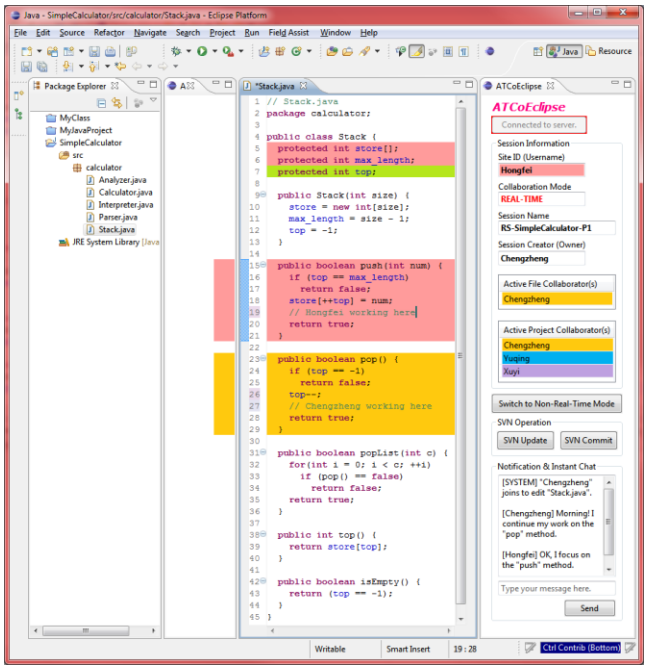


**Figure 7. Main user interface of the ATCoEclipse Client.**

In the middle panel, the source code editor looks almost the same as the single-user Eclipse source code editor, but many advanced real-time collaboration features have been embedded. Firstly, the local programmer's changes on the textual content of the source code file are instantly propagated to other active members in the same file session, and editing performed by other programmers in the same file session can also be noticed at this site in real-time. While programmers are concurrently editing the same source code file in a flexible and unconstrained way, consistency of the source code is maintained by underlying techniques at all times. Secondly, advanced collaboration awareness techniques devised in prior work [9] have been applied in the source code editor. As illustrated, workspaces of different programmers are highlighted by different background colors, so each programmer can intuitively see where and what others are doing in real-time, thus improving the communication and understanding among collaborators. In addition to workspace awareness, ATCoEclipse also incorporates novel techniques related to dependency relationships among source code regions (e.g., methods/fields of the Java class) contributed in prior work [8], which are applied to automatically derive dependency relationships among these regions (e.g., a method invokes another method, a method references a field) and highlight depended regions of programmers' working regions. For example, the local programmer (*Hongfei*) is working on the *push* method which references three fields (*int store[]*, *int max_length*, *int top*) of the class, and therefore all these regions are highlighted by the color assigned to the local user. Similarly, it can be watched that another collaborator (*Chengzheng*) is working on the *pop* method, which references one field (*int top*). To differentiate the working region and depended regions with respect to a programmer, a color bar is displayed to the left of the working region. In particular, the field *int top* is highlighted by a special color to indicate that it is currently referenced by both working regions of the two programmers, and they should pay attention to potential conflicts among their work. With these awareness features, programmers not only see where others are working, but also intuitively understand the relationships among collaborators' working regions to help avoid producing incompatible programming work.

In the right panel of the ATCoEclipse Client, important collaboration-related information is displayed, including the current collaboration mode (real-time/non-real-time), the session name, and the owner of the session. More importantly, the lists of collaborators in the same file session and the same project session are displayed respectively, which are dynamically updated when a new client joins the session or an existing active site leaves the session. A unique color is assigned to each collaborator in the lists, which is used for highlighting the working region and depended regions in the source code editor, as introduced above.

In the lower part of the right panel, the programmer may click a button to switch between real-time and non-real-time collaboration modes, thus achieving seamless transition among different collaboration modes. The version control operations (SVN *update* and SVN *commit*) are also provided for real-time collaborating programmers to enjoy close interaction with non-real-time collaborators. A notification box is placed below to deliver collaboration-related messages (e.g., who joins or leaves the session, who commits the latest source code copy to the SVN repository on behalf of the group). The notification box is also integrated with instant chatting features so that collaborating programmers can conduct real-time communication in case of need.

In summary, programmers can work with the ATCoEclipse system by using familiar functionalities and interface features that were available in the single-user Eclipse IDE, while in the meantime, enjoying novel any-time collaboration features.

## 5.3 Preliminary Performance Evaluation

The successful implementation of ATCoEclipse has validated the feasibility of ATCoPE. Preliminary performance evaluation has confirmed that the ATCoEclipse system achieves high performance in several aspects. Firstly, the local responsiveness of the client is as good as the single-user Eclipse IDE due to the repli-

cated architecture for the shared source code copy. Secondly, the propagation of editing operations to remote collaborating sites has been very efficient as well because most communication messages (e.g., editing operations, join-protocol messages) are less than 1kB in size, which can reach inter-continental remote sites normally within a delay of 100ms. Thirdly, a global synchronization procedure (e.g., join-protocol) involves a maximum of three single-trip messages, which can be transmitted within a total delay of less than 300ms (100ms for each single-trip). Fourthly, transmission of the entire source code copy for a project can be costly, depending on the total size of the project, but this occurs only when a new user joins a real-time session or a real-time collaborating programmer issues a non-real-time version control operation (*update* or *commit*) on behalf of the group, which is infrequent. Last but not least, ATCoEclipse is scalable, which is able to accommodate a large number of collaborating programmers for large-scale software projects, because: (1) most of the heavy work in either real-time or non-real-time collaboration sessions can be completed at individual and separated ATCoEclipse Clients; and (2) the ATCoEclipse Server has been designed and implemented to maintain administrative information and data only for active real-time sessions, without involvement in keeping track of non-real-time sessions and users or in performing real-time work, except for relaying real-time editing operations and communication messages via the session manager.

# 6. CONCLUSIONS AND FUTURE WORK

In this paper, we have contributed a novel *Any-Time Collaborative Programming Environment* (ATCoPE) in seamlessly integrating and supporting both real-time and non-real-time collaborative programming. Major contributions include the ATCoPE system architecture, functional design for ATCoPE, technical solutions for realizing ATCoPE, and the design and implementation of the *ATCoEclipse* prototype system as a proof-of-concept for ATCoPE. The prototype implementation and preliminary performance evaluation have confirmed the technical feasibility of ATCoPE and its supporting techniques, and provided positive feedback on the performance and scalability of the system. This research work has initiated a new direction in the domain of collaborative software development, and laid foundations for exploration of a range of interesting issues. Our ongoing work focuses on extending the functionalities of the ATCoEclipse system for conducting usability study, which will be reported in future papers.

# 7. REFERENCES


[1] Allen, L., Fernandez, G., Kane, K., Leblang, D. B., Minard, D., and Posner, J. ClearCase MultiSite: Supporting Geographically-Distributed Software Development. In *Software Configuration Management: Selected Papers from ICSE SCM-4 and SCM-5 Workshops, LNCS*, 1005 (1995), 194-214.

[2] Berliner, B. CVS II: Parallelizing Software Development. In *Proc. of USENIX Winter 1990 Technical Conf.* (1990), 341-352.

[3] Blackburn, J. D., Scudder, G. D., and Van Wassenhove, L. N. Improving Speed and Productivity of Software Development: A Global Survey of Software Developers. *IEEE Trans. Softw. Eng.* 22, 12 (1996), 875-885.

[4] Blackburn, J., Scudder, G., and Van Wassenhove, L. N. Concurrent software development. *Commun. ACM* 43, 11es, Article 4 (2000).

[5] Brooks, F. P. *The Mythical Man-Month (Anniversary Ed.)*. Addison-Wesley (1995).

[6] Cockburn, A. and Williams, L. The costs and benefits of pair programming. In *Extreme Programming Examined*, Addison-Wesley (2001), 223-243.

[7] Collins-Sussman, B. The subversion project: buiding a better CVS. *Linux J.* 2002, 94 (2002), 3.

[8] Fan, H. and Sun, C. Dependency-based automatic locking for semantic conflict prevention in real-time collaborative programming. In *Proc. of ACM Symposium on Applied Computing* (2012), 737-742.

[9] Fan, H. and Sun, C. Achieving integrated consistency maintenance and awareness in real-time collaborative programming environments: the CoEclipse approach. In *Proc. of IEEE Intl. Conf. on Computer Supported Cooperative Work in Design* (2012), 94-101.

[10] Goldman, M., Little, G., and Miller, R. C. Real-time collaborative coding in a web IDE. In *Proc. of ACM Symposium on User interface Software and Technology* (2011), 155-164.

[11] Loeliger, J. *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development (1st ed.)*. O'Reilly Media (2009).

[12] Nosek, J. T. The case for collaborative programming. *Commun. ACM* 41, 3 (1998), 105-108.

[13] Perry, D., Siy, H., and Votta, L. Parallel changes in large-scale software development: an observational case study. *ACM Trans. Softw. Eng. Methodol.* 10, 3 (2001), 308-337.

[14] Pilato, C., Collins-Sussman, B., and Fitzpatrick, B. *Version Control with Subversion (2 ed.)*. O'Reilly Media (2008).

[15] Salinger, S., Oezbek, C., Beecher, K., and Schenk, J. Saros: an eclipse plug-in for distributed party programming. In*Proc. of ICSE Workshop on Cooperative and Human Aspects of Softw. Eng.* (2010), 48-55.

[16] Shen, H. and Sun., C. Flexible notification for collaborative systems. In *Proc. of ACM Conf. on CSCW* (2002), 77-86.

[17] Shen, H. and Sun, C. Recipe: a web-based environment for supporting real-time collaborative programming. In *Proc. of Intl. Conf. on Networks, Parallel and Distributed Processing* (2002), 283-288.

[18] Sun, C. and Ellis, C. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *Proc. of ACM Conf. on CSCW* (1998), 59-68.

[19] Sun, C., Xia, S., Sun, D., Chen, D., Shen, H., and Cai, W. Transparent adaptation of single-user applications for multi-user real-time collaboration. *ACM Trans. Comput.-Hum. Interact.* 13, 4 (2006), 531-582.

[20] Tichy, W. F. Tools for software configuration management. In *Proc. of Intl. Workshop on Software Version and Configuration Control* (1988), 1-20.

[21] Williams, L. A. and Kessler, R. R. All I really need to know about pair programming I learned in kindergarten. *Commun. ACM* 43, 5 (2000), 108-114.

[22] Williams, L., Kessler, R. R., Cunningham, W., and Jeffries, R. Strengthening the case for pair programming. *IEEE Softw.* 17, 4 (2000), 19-25.